Alimentación Planta

Estado

La API persiste planificaciones en la colección `alimentaciones_planta`. El circuito es `pendiente -> aprobada -> activa -> finalizada`, con estado temporal `bloqueada` y salida `cancelada`. Solo una alimentación puede estar `activa` o `bloqueada`.

La identidad exacta de una tarjeta es:

```
{
  "nombre": "BE_160-75",
  "producto": "Mineral de Carles",
  "tipo": "mineral",
  "toneladasPlanificadas": 350
}
```

Al crear una planificación, cada tarjeta debe seguir existiendo en `bascula` como Acopio Stock Mineral: mismo `Area_Nivel`, mismo `Producto` y un grupo `estado-stock` con `tipo: "mineral"`.

Las tarjetas de `ronda` requieren `toneladasPlanificadas`, un número finito mayor que cero; se reservan contra el stock y determinan el mineral limitante y la duración nominal. Las tarjetas de `hora` requieren `entradasPorHora` y `toneladasPlanificadas`, se reservan contra el stock y participan en saldos y progreso. Una misma tarjeta no puede aparecer dos veces dentro de una alimentación ni estar simultáneamente en ambas listas.

Listar

`GET /alimentacion-planta/alimentaciones`

Devuelve `200` y las planificaciones ordenadas ascendentemente por `orden`.

```
[
  {
    "id": "66ba0000000000000000001",
    "codigo": "AL-20260810-AB12",
    "estado": "pendiente",
    "creadaEn": "2026-08-10T12:00:00.000Z",
    "aprobadaEn": null,
    "ronda": [
      {
        "nombre": "BE_160-75",
        "producto": "Mineral de Carles",
        "tipo": "mineral",
        "toneladasPlanificadas": 350
      }
```

```
    ],
    "hora": [
      {
        "nombre": "BE_160-76",
        "producto": "Mineral de Boinas",
        "tipo": "mineral",
        "entradasPorHora": 3,
        "toneladasPlanificadas": 42
      }
    ],
    "velocidadEstimadaTph": 42.5,
    "ciclosAlimentacionHora": 3,
    "cargaPalaEstimadaTon": 14,
    "caudalEfectivoTph": 42.5,
    "orden": 1
  }
]
```

Crear

`POST /alimentacion-planta/alimentaciones`

```
{
  "codigo": "AL-20260810-AB12",
  "ronda": [
    {
      "nombre": "BE_160-75",
      "producto": "Mineral de Carles",
      "tipo": "mineral",
      "toneladasPlanificadas": 350
    }
  ],
  "hora": [
    {
      "nombre": "BE_160-76",
      "producto": "Mineral de Boinas",
      "tipo": "mineral",
      "entradasPorHora": 3,
      "toneladasPlanificadas": 42
    }
  ],
  "velocidadEstimadaTph": 42.5,
  "ciclosAlimentacionHora": 3,
  "cargaPalaEstimadaTon": 14,
  "caudalEfectivoTph": 42.5,
  "insertarDespuesDeId": null,
  "planificacionPadreId": null
}
```

Ejemplo de tarjeta `hora`:

```
{
  "nombre": "BE_160-76",
  "producto": "Mineral de Boinas",
  "tipo": "mineral",
  "entradasPorHora": 3,
  "toneladasPlanificadas": 42
}
```

Para documentos nuevos, `hora[*].toneladasPlanificadas` es obligatorio y `ronda[*].entradasPorHora` es inválido. La API calcula las toneladas de cada tarjeta `hora` como $min(stockDisponibleToneladas, floor(duracionNominalHoras \times entradasPorHora) \times cargaPalaEstimadaTon)$ y exige que el valor recibido coincida con una tolerancia de 0,01 t. Las tarjetas históricas sin toneladas se migran al inicio; la migración falla claramente si no hay stock fiable para reconstruirlas.

Responde `201` con la planificación completa en estado `pendiente`. `velocidadEstimadaTph`, `cargaPalaEstimadaTon` y `caudalEfectivoTph` son obligatorios, finitos y mayores que cero; `ciclosAlimentacionHora` debe ser entero y mayor o igual que uno. Se exige $caudalEfectivoTph = ciclosAlimentacionHora \times cargaPalaEstimadaTon$. `insertarDespuesDeId` introduce el plan en esa posición; sin él se inserta al final. Un `planificacionPadreId` crea una subplanificación con su propio ID.

Errores: `422` para body inválido; `404` si una tarjeta no está disponible en Stock Mineral; `409` si el código existe o alguna tarjeta ya está asignada a una planificación pendiente o aprobada.

Aprobar

`POST /alimentacion-planta/alimentaciones/{alimentacion_id}/aprobar`

Responde `200`:

```
{
  "id": "66ba0000000000000000001",
  "estado": "aprobada",
  "aprobadaEn": "2026-08-10T12:15:00.000Z"
}
```

Solo aprueba una planificación `pendiente`. Una inexistente devuelve `404`; una ya aprobada devuelve `409`.

Circuito operativo

- `POST /alimentacion-planta/alimentaciones/{id}/activar`: solo una `aprobada`, y solo si no existe otra activa o bloqueada.
- `POST /alimentacion-planta/alimentaciones/{id}/bloquear`: solo una `activa`. Puede recibir `{ "observacion": "..." }`.

- `POST /alimentacion-planta/alimentaciones/{id}/desbloquear`: solo una `bloqueada`. Puede recibir la misma observación opcional.
- `POST /alimentacion-planta/alimentaciones/{id}/cancelar`: solo una `activa`. Requiere `{ "observaciones": "..." }`, conserva la observación y activa automáticamente la siguiente aprobada según `orden`.
- `POST /alimentacion-planta/alimentaciones/{id}/copiar`: admite una alimentación aprobada, activa, bloqueada o finalizada. Usa el body de creación, crea una subplanificación `pendiente` y asigna el origen como `planificacionPadreId`.

`GET /alimentacion-planta/circuito/actual` devuelve exclusivamente la alimentación activa o bloqueada, su mineral limitante, progreso, duración nominal y duración real. El mineral limitante es la tarjeta de `ronda` con menor duración nominal, calculada como $toneladasPlanificadas / velocidadEstimadaTph$; el progreso se calcula contra sus descargas registradas. Devuelve `404` si no existe circuito activo.

Cinta 4

`GET /produccion/cinta-4/velocidad` devuelve:

```
{ "toneladasHora": 38.2, "fecha": "2026-08-10T12:00:00.000Z" }
```

Lee el tag PLC real `fields.toneladas_Bascula_Cinta_4`. Si no hay una lectura numérica, finita y positiva, responde `503`; nunca sustituye el valor por la velocidad estimada.

Registrar descarga de pala

`POST /alimentacion-planta/alimentaciones/{alimentacion_id}/descargas-pala`

```
{
  "nombre": "BE_160-75",
  "producto": "Mineral de Carles",
  "toneladas": 18.5
}
```

La alimentación debe existir y estar `activa`. La tarjeta debe pertenecer a sus listas `ronda` u `hora` y no puede superar sus toneladas planificadas restantes. `nombre` y `producto` se limpian con trim; `toneladas` debe ser numérico, finito y mayor que cero.

La API descuenta el peso en orden FIFO: consume primero las mediciones de `bascula` con `Fecha` más antigua. Solo modifica `Peso_Neto` y añade una auditoría embebida por cada medición consumida. Nunca permite un valor negativo.

También añade el siguiente registro al array `descargasPala` del documento de alimentación:

```
{
  "fecha": "2026-08-10T12:00:00.000Z",
  "nombre": "BE_160-75",
  "producto": "Mineral de Carles",
  "toneladas": 18.5
}
```

Respuesta `200`:

```
{
  "alimentacionId": "66ba0000000000000000001",
  "nombre": "BE_160-75",
  "producto": "Mineral de Carles",
  "toneladas": 18.5,
  "stockRestanteToneladas": 457,
  "progresoPorcentaje": 52.9,
  "alimentacionSiguienteActivadaId": null
}
```

Errores: `422` para body inválido; `404` para alimentación inexistente o tarjeta ausente de Stock Mineral; `409` para alimentación no activa, tarjeta ajena, toneladas planificadas agotadas, stock insuficiente o concurrencia durante el descuento. Al agotar el mineral limitante, la alimentación se finaliza y se activa la siguiente aprobada según `orden`.

MongoDB standalone garantiza atomicidad por cada medición descontada y por el documento de alimentación, pero no entre todos los documentos. La API valida el saldo antes de escribir y detiene la descarga ante cualquier actualización condicional fallida; no usa transacciones ni compensaciones automáticas.

Reordenar

`PUT /alimentacion-planta/alimentaciones/orden`

```
{
  "ids": [
    "66ba0000000000000000002",
    "66ba0000000000000000001"
  ]
}
```

`ids` debe incluir cada planificación existente exactamente una vez. La API actualiza `orden` desde `1` usando `bulk_write` y responde `200` con el mismo array. Si los IDs no representan exactamente el conjunto actual o hay concurrencia, responde `409`.

Persistencia

La creación, aprobación y reordenado no modifican documentos de `bascula`. Las descargas de pala solo reducen `Peso_Neto` de Stock Mineral y escriben trazabilidad embebida. La API no crea stock, tarjetas, copias de mediciones ni colecciones de descargas, y no usa transacciones MongoDB ni replica set. Los índices creados al arrancar son `codigo` único, `estado` y `orden`.

Pruebas

```
$env:ALIMENTACION_PLANTA_TEST_MONGO_URI = "mongodb://servidor-
mongo:27017/orovalle_alimentacion_test"
py -m unittest tests.test_alimentacion_planta
```

Sin esa variable, las pruebas se omiten.